API SECURITY RISK ANALYSIS REPORT

CYBER SECURITY TASK 3 – FUTURE INTERNS

PREPARED BY: MELVIN KUNJUMON

DATE : 12TH AUGUST 2026

1.EXECUTIVE SUMMARY

This report presents an API security risk analysis conducted as part Of the Cyber Security Internship program at Future Interns. The Analysis targeted jsonplaceholder, a public demo REST API commonly Used for testing and learning, in order to identify common API Security risks in a safe, legally unambiguous environment.

Using Postman, read-only GET requests were sent to multiple endpoints To assess authentication requirements, access control, rate limiting, And input validation handling. Four findings were identified: three Represent genuine security gaps (missing authentication, absent rate Limiting, and lack of ownership-based access control), while the Fourth confirms that the API correctly handles invalid input without Leaking internal error details.

This report translates these technical findings into business-relevant Risk levels and practical remediation guidance, in the same format a Saas security consultant would deliver to a client.

2. METHODOLOGY

The target selected for this analysis was JSONPlaceholder (jsonplaceholder.typicode.com), a public, purpose-built demo API designed for testing and educational use. No production or third-party APIs were tested, in line with responsible testing practices.

All testing was performed using Postman (web version), sending only read-only GET requests. The following areas were assessed:

1. Authentication — Whether endpoints returning personal data required
   any form of API key, token, or authorization header.

2. Access Control — Whether individual user records could be accessed
   without any ownership or session verification.

3. Rate Limiting — Whether the API restricted the number of requests
   from a single source within a short time period. This was tested
   by sending 15-20 rapid consecutive requests to the same endpoint
   and observing the response behavior.

4. Input Validation — Whether the API handled malformed input (a
   non-numeric ID) and valid-but-nonexistent input (a numeric ID with
   no matching record) safely, without crashing or leaking internal
   error details.

No exploitation, authentication bypass attempts, or high-volume/DoS testing was performed at any point.

3. FINDINGS

#FINDING 1:

- **No Authentication Required on User Data Endpoints:-**

- What was found:

GET requests to /users and /users/1 returned complete personal data
— including names, email addresses, and physical addresses
— with no API key, token, or authorization header required or present.

- Business Impact:

On a production system, this would mean any unauthenticated party could retrieve the entire user database without logging in, resulting in a full-scale data exposure.

- Risk Level: High

- Remediation:

Require a valid API key or authentication token on any endpoint that
returns personal or sensitive data. Reject unauthenticated requests with a 401 Unauthorized response.

#FINDING 2:

➢ No Rate Limiting:-

- What was found:

The same endpoint (/users/1) was sent 15-20 rapid consecutive requests with no throttling, blocking, or 429 (Too Many Requests) response observed at any point.

- Business Impact:

Without rate limiting, the API is vulnerable to automated data scraping and could be abused to overload the system or brute-force
other protected endpoints once authentication is added.

- Risk Level: Medium

- Remediation:

Implement rate limiting per IP address or per API key (e.g. a maximum number of requests per minute), returning a 429 status code once the limit is exceeded.

#FINDING 3:

➢ No Ownership or Access Control Verification:-

- What was found:

Individual user records (e.g. /users/1) can be requested and returned in full by anyone, with no check confirming the requester is authorized to view that specific record.

- Business Impact:

This is a Broken Object Level Authorization (BOLA) risk — one of the most common and serious API vulnerabilities. In a real system, this could allow any user to view or manipulate another user's private data simply by changing an ID in the URL.

- Risk Level: High

- Remediation:

Implement server-side authorization checks that verify the authenticated user has permission to access the specific record being requested, rather than relying solely on the record's ID being known.

FINDING 4:

➢ Input Validation — Handled Correctly (Positive Finding)

- What was found:

Requests using an invalid ID format (/users/abc) and a valid-format but nonexistent ID (/users/9999) both returned clean 404 Not Found responses with empty bodies. No internal error messages, stack traces, or database details were leaked in either case.

- Business Impact:

This is good practice. Leaking internal error details is a common way attackers gather information about a system's internal structure; this API avoids that risk.

- Risk Level:

None — included as confirmation that this area was tested and found to be handled safely.

- Remediation:

None required. Continue returning generic error responses for invalid or nonexistent input in any production implementation.

4. CONCLUSION

This analysis of a public demo API identified three genuine security risks — missing authentication, absent rate limiting, and lack of object-level access control — alongside one confirmed area of good practice in input validation handling.

The two High-risk findings (missing authentication and broken object-level authorization) represent the kind of vulnerabilities that, in a production SaaS environment, would constitute a critical data exposure risk and should be prioritized for remediation before any such API is exposed publicly.

These findings reflect common, well-documented risk categories from the OWASP API Security Top 10, underscoring that even simple, demo-purpose APIs illustrate the same fundamental security principles that apply to production systems handling real user data.